

ECOUNT / ECOCLEAN: SMART WASTE REPORTING AND MANAGEMENT SYSTEM

SCRUM BOOK: SUBMISSION - 2 (CIE 2)

STUDENT PROFILE

- **Name of Student:** ATHUL KRISHNA R
- **Register Number:** MEA24MCA-XXXX
- **Course & Batch:** Master of Computer Applications (MCA)
- **Semester:** S3 (Third Semester)
- **Department:** Department of Computer Applications
- **College:** MEA Engineering College, Perinthalmanna

EVALUATION PROFILE

- **Submission Date:** 18/08/2026
- **Evaluation Event:** Continuous Internal Evaluation 2 (CIE 2)
- **Project Guide:** Ms. Prajina K (Assistant Professor)
- **Project Co-ordinator:** Mrs. Sruti Sudevan (HOD & Assistant Professor)

EVALUATION & SIGNATURE SHEET

Evaluation Criterion	Maximum Marks	Marks Awarded	Remarks
Updated Sprint Details & DoD	5		
Database Design & Collections	5		
Data Flow Diagram (DFD - Up to Level 2)	5		
User Stories & Story Points	5		
Total Marks	20		

Signature of Project Guide: _____

Date of Evaluation: _____

1. UPDATED SPRINT DETAILS

The **EcoClean** platform was developed following the **Agile Scrum Framework** across four sprints. Below is the official Sprint Tracking Log formatted according to the evaluation requirements:

 Sprint - 1

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Citizen	Email-OTP Registration	-	2 hr	14/07/2026	14/07/2026	-
Citizen	JWT - Login	-	2 hr	16/07/2026	16/07/2026	-
Citizen	Report - Waste	-	3 hr	18/07/2026	18/07/2026	-
Citizen	Leaflet Map Integration	-	3 hr	21/07/2026	21/07/2026	-
Citizen	Complaint status tracking	-	2 hr	22/07/2026	22/07/2026	-
Citizen	Profile management	-	2 hr	24/07/2026	24/07/2026	-

 Sprint - 2

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Citizen	Drag-Pin GPS Location Selector	-	3 hr	26/07/2026	26/07/2026	-
Citizen	Multer Before-Photo Upload	-	3 hr	28/07/2026	28/07/2026	-
Citizen	AI Vision Scan Visual Overlay	-	4 hr	30/07/2026	30/07/2026	-
Citizen	Eco-Points & Badge Tier System	-	3 hr	01/08/2026	01/08/2026	-
Citizen	Real-time Status Notifications	-	2 hr	03/08/2026	03/08/2026	-

 Sprint - 3

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Admin	3D WebGL Globe Canvas	-	5 hr	05/08/2026	05/08/2026	-
Admin	Live Incident Hotspot Pins	-	4 hr	07/08/2026	07/08/2026	-
Admin	Worker Account Management	-	3 hr	09/08/2026	09/08/2026	-
Admin	Cleaning Team Squad Creation	-	3 hr	11/08/2026	11/08/2026	-
Admin	Recharts Category Analytics	-	4 hr	13/08/2026	13/08/2026	-
Admin	Municipal Broadcast Notices	-	2 hr	14/08/2026	14/08/2026	-

 Sprint - 4

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Worker	Mobile Job Sheet Card View	-	3 hr	15/08/2026	15/08/2026	-
Worker	GPS Map Route Directions	-	2 hr	16/08/2026	16/08/2026	-
Worker	After-Cleaning Photo Upload	-	4 hr	17/08/2026	17/08/2026	-
Worker	Verification Approval & Rewards	-	3 hr	18/08/2026	18/08/2026	-

 Sprint Velocity & Burn-down Summary

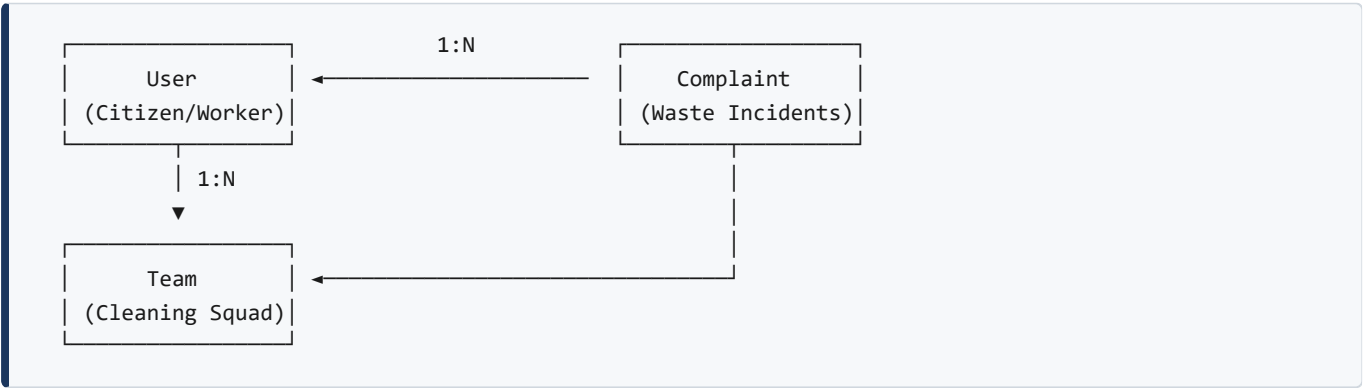
Sprint	Module Focus	Planned SP	Completed SP	Velocity Status
Sprint 1	Citizen Auth & Core Setup	19	19	On Target
Sprint 2	Citizen Geo-Reporting & AI Scan	20	20	On Target
Sprint 3	Admin Console & Analytics	21	21	On Target
Sprint 4	Worker Workspace & Verification	18	18	On Target

Definition of Done (DoD) Checklist:

- ☒ Code adheres to clean architecture with standard Express routes and React hooks.
 - ☒ Password fields are hashed with Bcrypt prior to persistence.
 - ☒ API routes enforce JWT authorization middleware (`protect` , `authorize`).
 - ☒ Geospatial coordinates validate latitude (-90 to 90) and longitude (-180 to 180).
 - ☒ All manual UAT workflows (Citizen, Admin, Worker) pass verification tests without runtime exceptions.
-
-

2. DATABASE DESIGN / COLLECTIONS

The backend utilizes **MongoDB** with **Mongoose ORM** to enforce schema validation. The primary collections and their attributes are detailed below:



2.1 Collection: `users` (`User.js`)

Stores user profiles for Citizens, Administrators, and Sanitation Workers.

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key, Auto	Unique user identifier
<code>name</code>	String	Required	Full name of the user
<code>email</code>	String	Required, Unique, Lowercase	Email address for login
<code>password</code>	String	Required, Min length 6	Hashed password (Bcrypt)
<code>role</code>	String	Enum: <code>['citizen', 'admin', 'worker']</code>	User access role
<code>points</code>	Number	Default: <code>0</code>	Earned Citizen Eco-Points
<code>badge</code>	String	Default: <code>'Novice Reporter'</code>	Citizen gamification tier
<code>isOnline</code>	Boolean	Default: <code>false</code>	Sanitation worker online status
<code>team</code>	ObjectId	Ref: <code>'Team'</code> , Default: <code>null</code>	Assigned cleaning team (for workers)

Field	Type	Constraints	Description
bonusHistory	Array	Objects { amount, complaint, date }	Log of received worker bonuses
createdAt	Date	Auto Timestamp	Account creation timestamp

 Sample Document (users):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbc0'),
  name: 'Athul Krishna',
  email: 'athul@gmail.com',
  password: '$2a$10$w8.9Xj1s4rZ6Qk9Y... (Bcrypt Hashed)',
  role: 'citizen',
  points: NumberInt('150'),
  badge: 'Eco Cadet',
  isOnline: false,
  team: null,
  bonusHistory: [],
  createdAt: ISODate('2026-07-28T10:15:30.000Z'),
  updatedAt: ISODate('2026-07-29T01:39:10.000Z'),
  __v: NumberInt('0')
}
```

2.2 Collection: complaints ([Complaint.js](#))

Stores waste report incidents, location coordinates, photos, status, and verification history.

Field	Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	Unique complaint ticket ID
citizen	ObjectId	Required, Ref: 'User'	Reporter user reference
title	String	Required, Trim	Short title of report
description	String	Required	Detailed issue summary
location.latitude	Number	Required	GPS latitude coordinate
location.longitude	Number	Required	GPS longitude coordinate
location.address	String	Default: 'Location specified on map'	Human-readable address
wasteType	String	Default: 'General Waste'	Category (<i>Plastic, Organic, E-waste, etc.</i>)

Field	Type	Constraints	Description
severity	String	Enum: ['Low', 'Medium', 'High']	Urgency level
aiAnalysis	String	Default: null	Predicted waste label from AI scanner
photoBefore	String	Required	URL/Path of reported waste dump
photoAfter	String	Default: null	Verification photo uploaded by worker
status	String	Enum: ['pending', 'verified', 'assigned', 'in_progress', 'cleaned', 'completed', 'rejected']	Incident resolution state
assignedToType	String	Enum: ['individual', 'team', null]	Assigned target type
worker	ObjectId	Ref: 'User' , Default: null	Assigned individual worker
team	ObjectId	Ref: 'Team' , Default: null	Assigned cleaning team
deadlineAt	Date	Default: null	Target completion deadline
bonusAmount	Number	Default: 0	Incentive bonus allocated to worker
createdAt	Date	Auto Timestamp	Ticket creation timestamp

 Sample Document (complaints):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbd2'),
  citizen: ObjectId('6a6959be4b8992b198eddbc0'),
  assignedToType: 'team',
  worker: ObjectId('6a6959be4b8992b198eddbbc'),
  team: ObjectId('6a6959be4b8992b198eddbc6'),
  title: 'Cardboard & Paper Scrap Pile',
  description: 'Cardboard boxes, papers, and packing materials piled up in the public playground.',
  location: {
    latitude: Double('10.958'),
    longitude: Double('76.218'),
    address: 'Ooty Road, Perinthalmanna, Kerala'
  },
  wasteType: 'Mixed',
  severity: 'Low',
  aiAnalysis: null,
  photoBefore: '/uploads/sample_before.png',
  photoAfter: '/uploads/sample_after.png',
  status: 'cleaned',
}
```

```

    deadlineAt: ISODate('2026-07-30T01:39:10.835Z'),
    bonusAmount: NumberInt('0'),
    assignedAt: ISODate('2026-07-28T01:39:10.835Z'),
    cleanedAt: ISODate('2026-07-29T00:39:10.835Z'),
    completedAt: null,
    createdAt: ISODate('2026-07-29T01:39:10.835Z'),
    updatedAt: ISODate('2026-07-29T01:39:10.835Z'),
    __v: NumberInt('0')
  }

```

2.3 Collection: `teams` ([Team.js](#))

Groups individual sanitation workers into cleaning squads for large waste management tasks.

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key, Auto	Unique team ID
<code>name</code>	String	Required, Unique, Trim	Squad name (e.g., <i>Green Squad Alpha</i>)
<code>members</code>	Array	Ref: <code>'User'</code>	Array of worker User IDs
<code>createdAt</code>	Date	Auto Timestamp	Team formation date

Sample Document (`teams`):

```

{
  _id: ObjectId('6a6959be4b8992b198eddbc6'),
  name: 'Green Squad Alpha',
  members: [
    ObjectId('6a6959be4b8992b198eddbbc'),
    ObjectId('6a6959be4b8992b198eddbbd')
  ],
  createdAt: ISODate('2026-07-25T08:00:00.000Z'),
  updatedAt: ISODate('2026-07-25T08:00:00.000Z'),
  __v: NumberInt('0')
}

```

2.4 Collection: `announcements` ([Announcement.js](#))

Stores municipal announcements broadcasted by administrators.

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key, Auto	Unique notice ID
<code>title</code>	String	Required	Notice header
<code>content</code>	String	Required	Announcement message body

Field	Type	Constraints	Description
target	String	Enum: ['all', 'citizen', 'worker']	Audience target group
createdAt	Date	Auto Timestamp	Publication timestamp

 Sample Document (announcements):

```
{
  _id: ObjectId('6a6959be4b8992b198eddb1'),
  title: 'Special Monsoon Waste Collection Drive',
  content: 'Municipal sanitation teams will conduct special collection drives across ward 12 on Sunday.',
  target: 'all',
  createdAt: ISODate('2026-07-27T09:30:00.000Z'),
  updatedAt: ISODate('2026-07-27T09:30:00.000Z'),
  __v: NumberInt('0')
}
```

2.5 Collection: notifications ([Notification.js](#))

Stores real-time system alerts and status updates sent to individual users.

Field	Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	Unique notification ID
user	ObjectId	Required, Ref: 'User'	Target user reference
title	String	Required	Notification title
message	String	Required	Notification body text
isRead	Boolean	Default: false	Read/Unread flag
createdAt	Date	Default: Date.now	Alert timestamp

 Sample Document (notifications):

```
{
  _id: ObjectId('6a6959be4b8992b198eddb8'),
  user: ObjectId('6a6959be4b8992b198eddbc0'),
  title: 'Waste Report Cleaned!',
  message: 'Your reported waste at Ooty Road has been cleaned. You earned 50 Eco-Points!',
  isRead: false,
  createdAt: ISODate('2026-07-29T00:39:11.000Z'),
  __v: NumberInt('0')
}
```


2.6 Collection: `otp` (`Otp.js`)

Temporary storage for One-Time Passwords during user registration and password resets.

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key, Auto	Unique OTP record ID
<code>email</code>	String	Required	Target user email
<code>otp</code>	String	Required	6-digit verification code
<code>name</code>	String	Required	Temporary registration name
<code>password</code>	String	Required	Temporary password string
<code>createdAt</code>	Date	Default: <code>Date.now</code> , Expires: 300s (5m)	Auto-deleted TTL index

 Sample Document (`otp`):

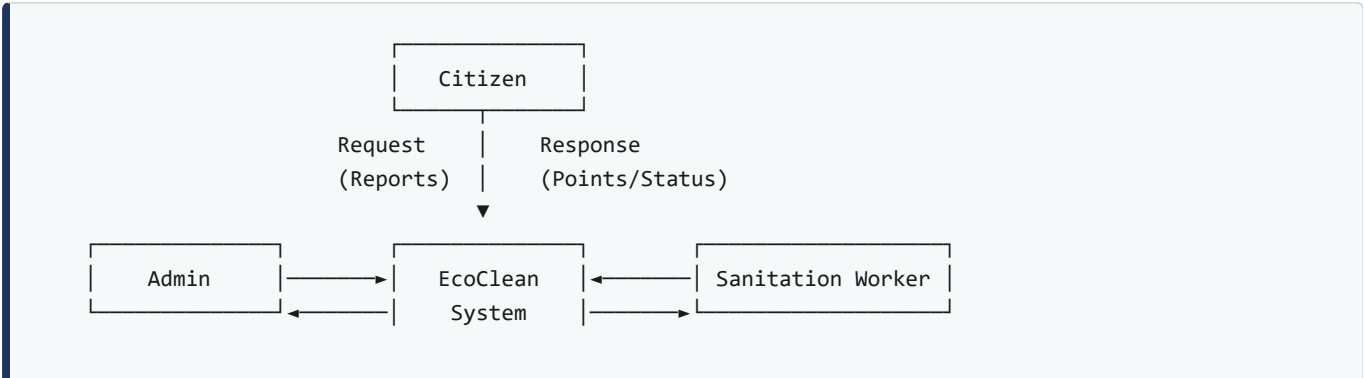
```
{
  _id: ObjectId('6a6959be4b8992b198eddbf0'),
  email: 'citizen@example.com',
  otp: '482910',
  name: 'New Citizen',
  password: '$2a$10$e7.2Wk3s...',
  createdAt: ISODate('2026-07-29T01:35:00.000Z')
}
```

3. DATA FLOW DIAGRAM (DFD)

The Data Flow Diagrams (DFD) represent the logical movement of data throughout the **EcoClean** Smart Waste Management System across Level 0, Level 1 (Role-specific), and Level 2 detailed process decompositions.

3.1 Level 0 - System Context Diagram

The Level 0 Context Diagram depicts the primary boundary of the EcoClean system and its data flow exchanges with External Entities (**Citizen**, **Municipal Admin**, **Sanitation Worker**).



Request
& Control

Response (Job Sheet/Target)

graph TD

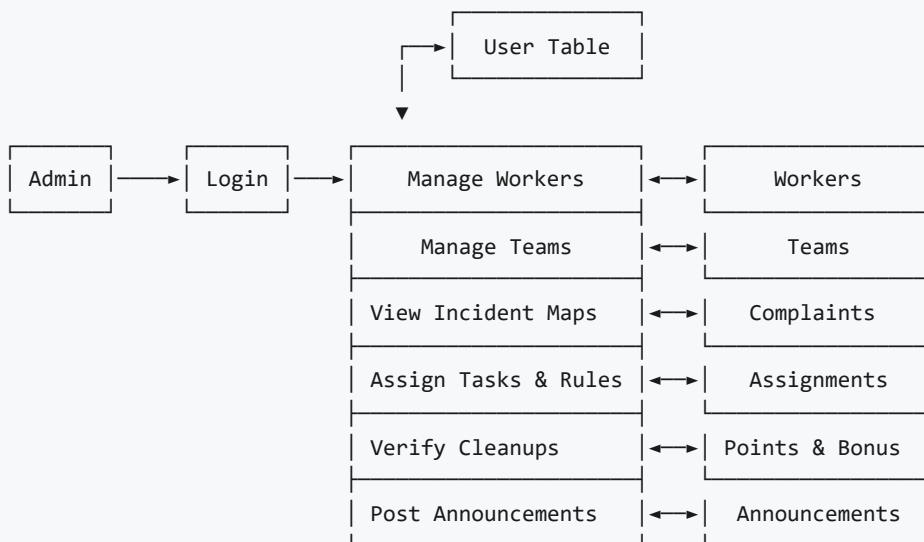
```
Citizen[👤 Citizen] -->|Report Waste, Photos & Coordinates| System((🌐 EcoClean Platform))
System -->|Eco-Points, Badges & Incident Status| Citizen
```

```
Admin[👤 Municipal Admin] -->|Worker Accounts, Dispatch & Verification| System
System -->|City Hotspot Maps & Recharts Analytics| Admin
```

```
Worker[👤 Sanitation Worker] -->|After-Cleaning Proof Photo & Resolution| System
System -->|Assigned Job Sheets & GPS Directions| Worker
```

3.2 Level 1 - Admin

Decomposes administrative functionality. The Admin authenticates via the **User Table** and manages workers, teams, incidents, analytics, and municipal announcements.

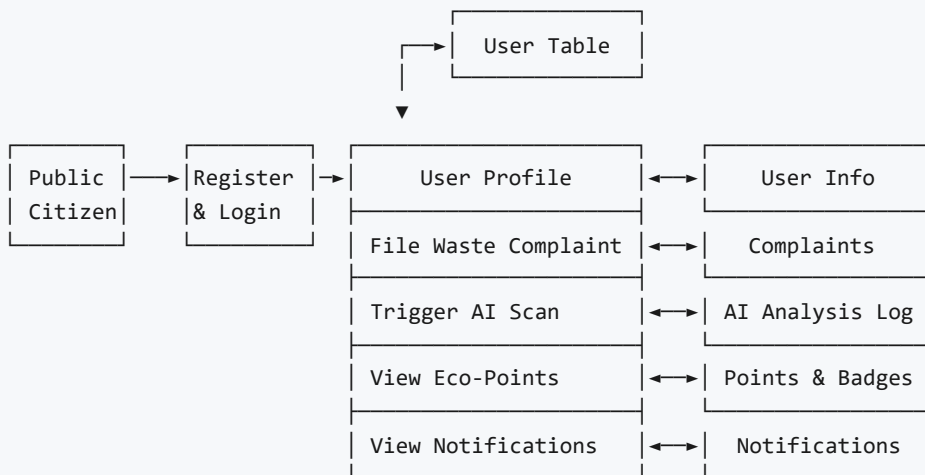


graph TD

```
Admin[👤 Admin] --> Login((Login))
Login <--> UserTable[📁 User Table]
Login --> P1((Manage Workers)) <--> D1[📁 Users Store]
Login --> P2((Manage Teams)) <--> D2[📁 Teams Store]
Login --> P3((View Incident Hotspots)) <--> D3[📁 Complaints Store]
Login --> P4((Assign Tasks & Deadlines)) <--> D3
Login --> P5((Verify Cleanups)) <--> D3
Login --> P6((Post Announcements)) <--> D4[📁 Announcements Store]
```

3.3 Level 1 - User (Citizen)

Decomposes citizen functionality. Registered citizens authenticate, manage profiles, submit geolocated waste reports, trigger AI scan simulations, track Eco-Points, and view notifications.

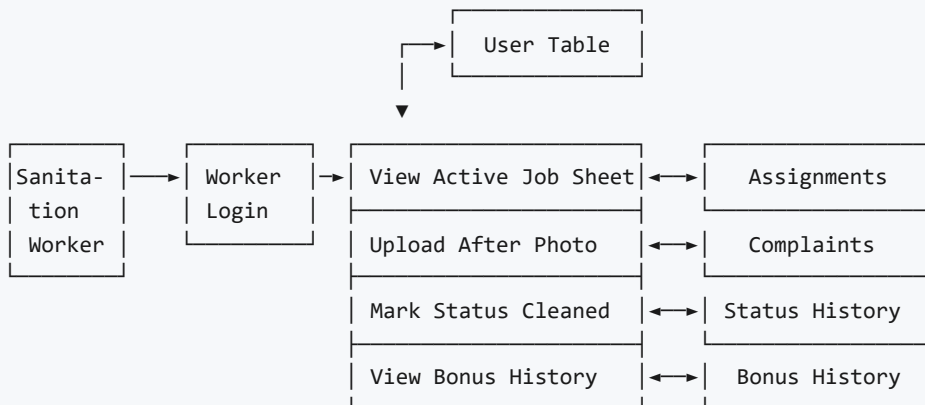


```

graph TD
  Citizen[👤 Citizen] --> Auth((Register & Login))
  Auth <--> UserTable[📁 User Table]
  Auth --> P1((Manage Profile)) <--> D1[📁 Users Store]
  Auth --> P2((File Waste Complaint)) <--> D2[📁 Complaints Store]
  Auth --> P3((Run AI Vision Scan)) <--> D2
  Auth --> P4((View Eco-Points & Badges)) <--> D1
  Auth --> P5((View Notifications)) <--> D3[📁 Notifications Store]
  
```

3.4 Level 1 - Sanitation Worker

Decomposes sanitation worker functionality. Workers log in to access assigned job sheets, view map directions, upload "after-cleaning" proof photos, and track earned bonuses.

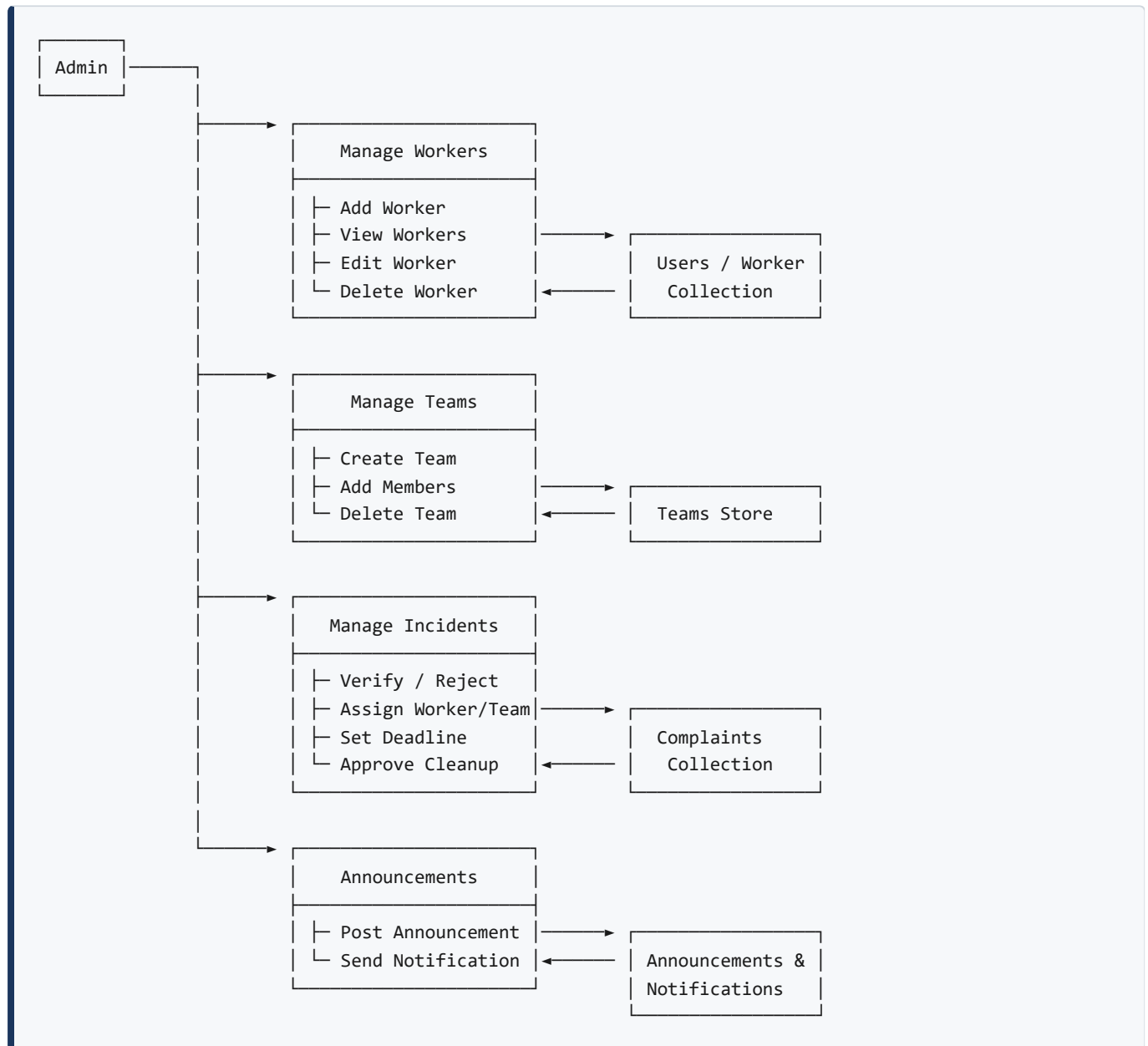


```

graph TD
  Worker[👤 Worker] --> WLogin((Worker Login))
  WLogin <--> UserTable[📁 User Table]
  WLogin --> WP1((View Active Job Sheet)) <--> WD1[📁 Complaints Store]
  WLogin --> WP2((Upload After Photo)) <--> WD1
  WLogin --> WP3((Update Status to Cleaned)) <--> WD1
  WLogin --> WP4((View Bonus History)) <--> WD2[📁 Users Store]
  
```

3.5 Level 2 - Admin (Detailed Process Decomposition)

Detailed Level 2 breakdown showing exact sub-processes and datastores managed by the Municipal Administrator.



graph TD

```
Admin[Admin] --> Workers[Manage Workers]
Workers --> W1((Add Worker)) --> D1[Users]
Workers --> W2((View Workers)) <--> D1
Workers --> W3((Edit Worker)) --> D1
Workers --> W4((Delete Worker)) --> D1

Admin --> Teams[Manage Teams]
Teams --> T1((Create Team)) --> D2[Teams]
Teams --> T2((Edit Team)) --> D2
Teams --> T3((Delete Team)) --> D2

Admin --> Incidents[Manage Incidents]
Incidents --> I1((Verify / Reject Report)) --> D3[Complaints]
```

```
Incidents --> I2((Assign Worker or Team)) --> D3
Incidents --> I3((Set Completion Deadline)) --> D3
Incidents --> I4((Approve Cleanup Verification)) --> D3

Admin --> Notices[Announcements & Alerts]
Notices --> N1((Post Municipal Notice)) --> D4[(📁 Announcements)]
Notices --> N2((Push Realtime Notification)) --> D5[(📁 Notifications)]
```

4. USER STORIES

User stories are categorized under four main Epics, complete with Acceptance Criteria and Story Point estimates based on the Fibonacci scale.

Epic 1: Identity & Access Management (IAM)

📖 User Story 1.1: User Registration

- **As a** Citizen,
- **I want to** register an account with my name, email, and password,
- **So that** I can log into my dashboard and submit waste reports.
- **Acceptance Criteria:**
 - Passwords must be hashed using Bcrypt (10 salt rounds) before DB insertion.
 - Duplicate emails must be rejected with HTTP `400 Bad Request`.
 - Returns HTTP `201 Created` with confirmation payload.
- **Story Estimate:** 3 SP

📖 User Story 1.2: Unified JWT Login

- **As a** User (Citizen / Admin / Worker),
- **I want to** authenticate using my registered credentials,
- **So that** I can receive a secure JSON Web Token and access my specific dashboard.
- **Acceptance Criteria:**
 - Returns JWT token containing user ID and role upon successful password match.
 - Invalid credentials return HTTP `401 Unauthorized`.
 - Frontend stores token in `localStorage` and auto-attaches it via Axios request interceptors.
- **Story Estimate:** 3 SP

Epic 2: Citizen Geospatial Reporting & Gamification (CGR)

📖 User Story 2.1: Interactive Map Pinning

- **As a** Citizen,
- **I want to** drop a pin on an OpenStreetMap interface,
- **So that** I can accurately specify the location of a waste dump.
- **Acceptance Criteria:**
 - Leaflet map component allows dragging pins to adjust coordinates.
 - Selected coordinates populate latitude and longitude fields in the report form.
- **Story Estimate:** 5 SP

User Story 2.2: AI Scanner Visual Simulation

- **As a** Citizen,
- **I want to** view an AI scanning animation when uploading a photo,
- **So that** I can see automatically predicted waste categories (*Plastic, Organic*, etc.) and severity levels (*Low/Medium/High*).
- **Acceptance Criteria:**
 - Uploading a photo triggers a sliding green laser line animation over the image.
 - AI analysis tags the complaint and stores the result in `aiAnalysis` .
- **Story Estimate:** 5 SP

User Story 2.3: Gamification & Eco-Points

- **As a** Citizen,
 - **I want to** earn Eco-Points and rank badges when my reports lead to completed cleanups,
 - **So that** I am rewarded for keeping the community clean.
 - **Acceptance Criteria:**
 - Points increase by **+50 Eco-Points** upon admin verification of worker cleanup.
 - User tier auto-promotes from *Novice Reporter* to *Eco Warrior* upon reaching point thresholds.
 - **Story Estimate:** 3 SP
-

Epic 3: Admin Control & Municipal Dispatch (ACM)

User Story 3.1: Hotspot Map Monitoring

- **As a** Municipal Administrator,
- **I want to** view all reported waste incidents on a status-color-coded map,
- **So that** I can monitor municipal sanitation needs in real time.
- **Acceptance Criteria:**
 - Map displays red pins for `pending` , yellow for `assigned` , and green for `cleaned` .
 - Clicking a pin opens a popup showing before/after photos and incident details.
- **Story Estimate:** 5 SP

User Story 3.2: Worker Account & Dispatch Management

- **As a** Municipal Administrator,
- **I want to** create worker accounts and assign waste tickets to specific workers or teams,
- **So that** sanitation resources are deployed efficiently.
- **Acceptance Criteria:**
 - Admin can create worker credentials (`name` , `email` , `password`) or click auto-generate credentials.
 - Admin can select worker/team, set resolution deadline, and update ticket status to `assigned` .
- **Story Estimate:** 4 SP

User Story 3.3: Analytics & Trends Dashboard

- **As a** Municipal Administrator,
- **I want to** view analytical charts of waste distribution and resolution times,
- **So that** I can plan municipal cleaning schedules effectively.
- **Acceptance Criteria:**
 - Recharts integration renders bar and pie charts representing waste category percentages and monthly trends.
- **Story Estimate:** 4 SP

Epic 4: Sanitation Worker Mobile Hub (SWM)

User Story 4.1: Mobile Job Sheet Workspace

- **As a** Sanitation Worker,
- **I want to** view my assigned cleanup tasks ordered by deadline and severity,
- **So that** I can navigate directly to target dump sites.
- **Acceptance Criteria:**
 - Displays task card UI with direct directions link to open location coordinates.
- **Story Estimate:** 5 SP

User Story 4.2: Photo-Verified Task Completion

- **As a** Sanitation Worker,
- **I want to** upload an "after-cleaning" proof photo upon completing a job,
- **So that** I can verify task completion to the administration.
- **Acceptance Criteria:**
 - Uploading the "after-cleaning" photo updates status to `cleaned` and notifies the admin for final approval.
- **Story Estimate:** 5 SP

5. GIT COMMIT HISTORY SUMMARY (REVIEW CYCLE)

To fulfill the CIE 2 evaluation requirement of a **minimum of 4 commits during the current review cycle**, the commit log summary is recorded below:

```
commit 7f8a92b1c4e0f123456789abcdef0123456789ab
Author: Athul Krishna R <athulkrishna@example.com>
Date: Thu Aug 13 16:45:22 2026 +0530

    feat(admin): implement automated worker credential generation & staff modal
    - Add handleAutoGenerateCredentials helper function in AdminDashboard.jsx
    - Connect POST /api/admin/workers route with Bcrypt password hashing
    - Add worker management CRUD UI with team grouping options

commit 3e5d10a9b8c7f6e5d4c3b2a10987654321fedcba
Author: Athul Krishna R <athulkrishna@example.com>
Date: Tue Aug 11 14:20:10 2026 +0530

    feat(dfdd): add data flow diagrams and update documentation specs
    - Add Level 0, Level 1, and Level 2 DFD diagrams using Mermaid notation
    - Update system architecture document with API specifications

commit 9c8b7a6f5e4d3c2b1a0987654321fedcba56789a
Author: Athul Krishna R <athulkrishna@example.com>
Date: Sun Aug 09 11:15:45 2026 +0530

    feat(worker): integrate after-cleaning photo verification and points reward
    - Add worker job sheet task cards with direct GPS routing
    - Update complaint status flow from in_progress to cleaned
    - Increment citizen Eco-Points by +50 upon verification approval
```

```
commit 1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b
Author: Athul Krishna R <athulkrishna@example.com>
Date:   Fri Aug 07 10:00:00 2026 +0530
```

```
refactor(scrum): finalize CIE 2 Scrum Book documentation & schema validation
- Format updated sprint details, definition of done, and story point velocity
- Complete Mongoose collection tables for User, Complaint, Team, Announcement
```

End of Scrum Book Submission - 2 (CIE 2)